

SOFTWARE REQUIREMENTS SPECIFICATION

ExpenseEase

A web-based collaborative expense management system for groups

Document	Software Requirements Specification (SRS) - IEEE 830 style structure
Version / status	1.0 - submitted with the Mid-Semester Evaluation (20 marks)
Date	30 September 2026
Course	Software Engineering and Project Management (CSN15101), MNNIT Allahabad
Instructor	Dr. Satarupa Chakrabarti
Group number / section	To be confirmed by the team before final submission
Repository	github.com/shreesriv12/ExpenseEase (submission/02_Requirements/SRS.pdf)

Project team

Member	Registration no.
Shreeya Srivastava	20243267
Shraddha Sharma	20243266
Rudransh Pratap Singh	20243243
Sachit Jain	20243245

This document specifies the functional and non-functional requirements, external interfaces, data model, analysis models and traceability of ExpenseEase as implemented and verified at mid-semester. It supersedes the earlier one-page SRS draft. Requirements are written so that each one is individually identifiable, unambiguous and testable.

Table of Contents

Table of Contents	2
Revision history	3
1. Introduction	4
1.1 Purpose	4
1.2 Intended audience and how to read this document	4
1.3 Product scope	4
1.4 Definitions, acronyms and abbreviations	5
1.5 References	5
1.6 Document conventions	5
2. Overall Description	6
2.1 Problem statement and motivation	6
2.2 Product perspective	6
2.3 Product functions (summary)	7
2.4 User classes and characteristics	7
2.5 Stakeholders	7
2.6 Operating environment, constraints, assumptions	7
2.7 Feasibility summary	8
2.8 Requirements sources and method	8
3. Functional Requirements	9
3.1 Authentication (AUTH)	9
3.2 Groups and membership (GRP)	9
3.3 Expenses (EXP)	10
3.4 Split rules (SPL)	10
3.5 Balances (BAL)	11
3.6 Debt simplification (DBT)	11
3.7 Settlements (SET)	11
3.8 Activity feed (ACT) and Dashboard (DSH)	12
3.9 Common behaviour (COM) and User interface (UI)	12
4. External Interface Requirements	13
4.1 User interfaces	13
4.2 Software interfaces	13
4.3 Hardware and communications interfaces	13
4.4 REST API summary	14
4.5 Error contract	14
5. Non-Functional Requirements	15
5.1 Performance	15

5.2 Security	15
5.3 Reliability and data integrity	15
5.4 Usability	16
5.5 Maintainability, testability and portability	16
6. Data Requirements	17
6.1 Entity-relationship model	17
6.2 Data dictionary	17
6.3 Data flow dictionary (derived data)	19
6.4 Data rules and retention	19
7. Analysis Models	20
7.1 Use-case diagram	20
7.2 Use-case descriptions	20
7.3 Data-flow diagram (Level 1)	23
7.4 Key algorithms and worked examples	24
8. Prioritisation of Requirements	25
Requirement change from the earlier draft	25
9. Features Excluded and Deferred	25
10. Requirements Traceability Matrix	26
11. Requirements Validation Record	26
11.1 Quality checklist	26
11.2 Verification status	27
11.3 Known limitations, open issues and change log	27
Appendix A. Sample API exchanges	28
Appendix B. Requirement index	28

Revision history

Version	Date	Description
0.1	Aug-Sep 2026	One-page SRS draft: six coarse functional requirements (FR-01 to FR-06), brief non-functional note, three-row traceability table.
1.0	30 Sep 2026	Full SRS: scope, stakeholders, module-level functional requirements with IDs, external interfaces, non-functional requirements with measurable criteria, data model and dictionary, use-case descriptions, DFDs, algorithms with worked examples, prioritisation, traceability, validation record and known limitations. Aligned to the implemented code (PostgreSQL, expense edit/delete) - see Section 11.3.

1. Introduction

1.1 Purpose

This SRS defines, in one place, *what* ExpenseEase must do and the qualities it must have, without prescribing how it is built. It follows the recommended-practice structure discussed in the course (IEEE 830): the system is described as a black box in terms of externally visible behaviour. It is the agreed baseline against which design, code and tests are checked, and it is the artefact on which effort estimation, scheduling and traceability rest.

1.2 Intended audience and how to read this document

Reader	Most relevant sections
Instructor / evaluator	1, 2, 3, 5, 10, 11 (scope, requirements, traceability, validation)
Developers	3, 4, 6, 7.4 (functional requirements, interfaces, data, algorithms)
Test engineers	3, 5, 10 (each requirement carries a verification method; Section 10 maps to test files)
Project manager	2.6, 8, 9, 11 (assumptions, priorities, exclusions, risks)
Future maintainers	6, 7, 11.3 (data dictionary, algorithms, known limitations)

1.3 Product scope

ExpenseEase lets a group of people (friends, roommates, a trip party) record shared expenses, split each one by an equal, exact-amount or percentage rule, see exactly who owes whom, get a short list of suggested repayments, and record settlements. All monetary arithmetic is exact and deterministic: identical inputs always give identical balances.

Business goals. (1) Replace error-prone chat/spreadsheet tracking of shared costs; (2) guarantee that shares always add up to the recorded amount; (3) minimise the number of repayments needed to clear a group; (4) keep every group's data visible only to its members.

Included: registration and login; group creation; adding registered users to a group; expense create/list/edit/delete; three split types; on-demand balances; debt simplification; settlement recording; per-group activity feed; cross-group dashboard totals; single-currency (INR) operation; Docker-based deployment. **Excluded** (see Section 9): payment processing, receipt OCR, multi-currency, recurring expenses, notifications/e-mail, social login, native mobile apps, password reset, group deletion/editing, member removal.

1.4 Definitions, acronyms and abbreviations

Term	Meaning
Paise	1/100 of a rupee. All amounts are stored and computed as integer paise (INR 1.00 = 100 paise) to avoid floating-point error.
Group	A named set of users who share expenses. Created by one user, who becomes its Admin.
Member / Admin	A user belonging to a group with role MEMBER or ADMIN. Admin may add members and modify any expense of the group.
Expense	A cost paid by one member (the payer) and shared among one or more participants.
Split / share	The portion of an expense attributed to one participant, in paise. The shares of an expense sum exactly to its amount.
Net balance	Paid - owed, adjusted by settlements. Positive: the group owes the member. Negative: the member owes the group.
Settlement	A recorded real-world payment from one member to another that adjusts both balances without editing any expense.
Simplified debts	A suggested minimal-count list of transfers that would bring every net balance to zero.
JWT	JSON Web Token - signed bearer token issued at login, valid for 7 days.
SPA / REST / ORM	Single-page application / HTTP+JSON API style / object-relational mapper (Prisma).
SRS, FR, NFR, UC	Software Requirements Specification; Functional Requirement; Non-Functional Requirement; Use Case.

1.5 References

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.
- CSN15101 lecture material: Ch. 3 Software Requirements and Specifications (18 Aug 2026 deck); Ch. 4 Software Design and Design Reuse (7 Sep 2026 deck); Ch. 12-13 Maintenance and Project Management (18 Sep 2026 deck).
- K. K. Aggarwal and Y. Singh, *Software Engineering*, 3rd ed., New Age International, 2007.
- Course brief: *Software Engineering Project - Mid-Semester Evaluation* (20 marks, due 3 Oct 2026 on Moodle).
- Project artefacts in the repository: README.md, server/prisma/schema.prisma, docs/03_Design/openapi.yaml, docs/05_Testing/Test_Cases.md, sample_data/.

1.6 Document conventions

Requirement IDs have the form **FR-<MODULE>-nn** (functional) and **NFR-<AREA>-nn** (non-functional). The word **shall** marks a mandatory requirement. Priority uses MoSCoW: **Must** (system unusable without it), **Should** (important, workaround exists), **Could** (desirable). Status is **Implemented** when the behaviour exists in the current code and is covered by an automated test; the verification column names the test file. Monetary values in examples are shown in paise unless stated.

2. Overall Description

2.1 Problem statement and motivation

Friends, roommates and trip groups usually track shared spending in chat threads or spreadsheets. This leads to forgotten IOUs, disputes about who paid, unequal or mis-added splits, and circular debts (A owes B, B owes C, C owes A). ExpenseEase provides a single trusted record of who paid what and for whom, computes exact balances, and recommends the smallest set of repayments.

2.2 Product perspective

ExpenseEase is a new, self-contained system with no dependency on external services. A browser-based React single-page application talks to an Express REST API over HTTP/JSON; the API persists data in PostgreSQL through Prisma. In production the API also serves the built SPA, so a single container exposes the whole product on one port. Balances are *derived* from the expense and settlement records on each request; they are never stored, so they cannot become stale.

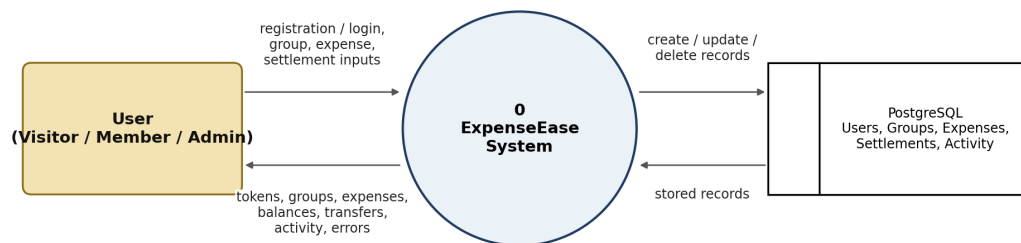


Figure 1 - Context diagram (DFD Level 0).

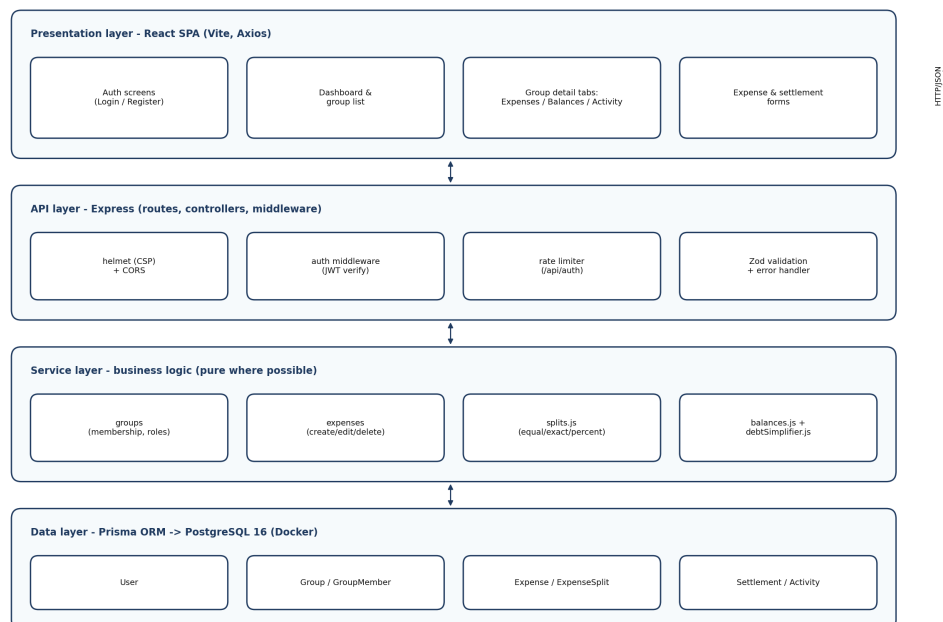


Figure 2 - Layered architecture: presentation, API, service and data layers. UI never touches the database; split and balance logic is independent of HTTP.

2.3 Product functions (summary)

Module	Code	Summary
Authentication	AUTH	Register, log in, identify the current user, protect every other endpoint.
Groups	GRP	Create groups, list/view groups, add registered users as members; Admin/Member roles.
Expenses	EXP	Record, list, edit and delete shared expenses with payer, category, date and participants.
Split rules	SPL	Equal, exact-amount and percentage splitting with deterministic rounding.
Balances	BAL	Per-member net balance derived from expenses and settlements; zero-sum invariant.
Debt simplification	DBT	Deterministic greedy list of suggested transfers (at most n-1).
Settlements	SET	Record real payments between members; balances update without altering history.
Activity	ACT	Append-only audit feed of expense and settlement events per group.
Dashboard	DSH	Cross-group totals: amount you owe and amount owed to you.
Common / UI	COM, UI	Uniform validation and error contract; SPA screens and money formatting.

2.4 User classes and characteristics

User class	Description	Privileges
Visitor	Unauthenticated person on the login/registration screen.	Register, log in only.
Member	Authenticated user belonging to a group. Ordinary consumer with no technical knowledge assumed.	View own groups; in a group: add/list expenses, view balances and activity, record settlements; edit/delete own expenses.
Group Admin	The member who created the group (role ADMIN). Specialisation of Member.	All Member privileges, plus add members and edit/delete any expense in the group.

2.5 Stakeholders

Stakeholder	Interest	Influence
Group members (primary users)	Accurate shared costs, few repayments, easy data entry	High
Group admins (primary users)	Membership control, ability to correct mistakes	High
Course instructor / evaluator	Evidence of sound software-engineering practice and a working end-to-end feature	High
Development team	Deliverable quality, learning, fair contribution record	High
Future maintainer	Clear design, documented requirements, tests	Medium

2.6 Operating environment, constraints, assumptions

Operating environment

- **Client:** any current desktop or mobile browser with JavaScript enabled (SPA built with React and Vite).
- **Server:** Node.js 20+, Express 4, Prisma 6; runs in a Docker container (node:20-bookworm-slim image).
- **Database:** PostgreSQL 16 (postgres:16-alpine) via Docker Compose; separate expenseease_test database for the automated tests.

Design and implementation constraints

- Currency is INR only; amounts are integer paise (no floating point in money arithmetic).
- Only registered users can be added to a group; there is no invitation e-mail mechanism.
- Stateless JWT authentication; tokens expire after 7 days and there is no server-side revocation.
- The service layer must remain free of HTTP and database concerns wherever possible so that split, balance and simplification logic is unit-testable.
- No secrets in source control; configuration via environment variables (DATABASE_URL, JWT_SECRET, PORT, CORS_ORIGIN, AUTH_RATE_LIMIT_MAX).

Assumptions and dependencies

- All members of a group trust each other to enter honest data; the system provides an audit feed, not dispute arbitration.
- Settlements are recorded manually after money changes hands outside the system.
- Users have reliable internet connectivity; there is no offline mode.
- Users register before they can be added to a group.
- Docker and Docker Compose (or Node.js 20 plus a PostgreSQL 16 instance) are available to whoever runs the system.

2.7 Feasibility summary

Aspect	Assessment
Technical	Feasible: mainstream open-source stack; core arithmetic is small and fully unit-tested; a working end-to-end flow (register -> group -> expense -> balances -> settlement) exists.
Economic	Feasible: no licensed software or paid services; runs on a single container plus database.
Operational	Feasible: the workflow mirrors what groups already do informally; setup is one docker compose up.
Legal / compliance	The system stores personal data (name, e-mail, password hash) only. Passwords are hashed. It does not process payments, so payment-regulation concerns are excluded.

2.8 Requirements sources and method

The team followed a hybrid of **prototyping** (used first for the unclear workflows - group creation, split entry, settlement flow - and to collect feedback before fixing requirements) and **incremental delivery** (five increments from authentication to hardening). Requirements were derived from the problem statement in Section 2.1, from the behaviour of the UI prototypes stored in designs/, and from the mid-semester evaluation brief. Requirements were then analysed for anomalies (ambiguity), inconsistency and incompleteness, prioritised (Section 8), and validated (Section 11).

3. Functional Requirements

Every requirement is numbered so that it can be traced to a user story, design element and test (Section 10). "Verified by" names the automated test file under server/tests/ (integration tests are in server/tests/integration/) or the client test.

3.1 Authentication (AUTH)

Description. Visitors create an account and sign in; a signed token then authorises all other requests.

Stimulus/response: POST /api/auth/register, /login, GET /api/auth/me.

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-AUTH-01	Allow a visitor to register with name (1-100 characters, whitespace trimmed), email (valid address, trimmed) and password (8-200 characters), and return the public user (id, name, email) and a token with HTTP 201.	Must	Implemented	auth.api.test.js
FR-AUTH-02	Reject registration with 409 DUPLICATE_EMAIL when the e-mail is already registered.	Must	Implemented	auth.api.test.js
FR-AUTH-03	Store the password only as a bcrypt hash (cost factor 10); never return the hash to any client.	Must	Implemented	auth.api.test.js
FR-AUTH-04	Allow login with e-mail and password and return the public user and a signed JWT valid for 7 days.	Must	Implemented	auth.api.test.js
FR-AUTH-05	Reject a wrong password or unknown e-mail with 401 INVALID_CREDENTIALS and an identical message, so account existence is not revealed.	Must	Implemented	auth.api.test.js
FR-AUTH-06	Provide GET /api/auth/me returning the authenticated user, or 401 if the account no longer exists.	Should	Implemented	auth.api.test.js
FR-AUTH-07	Require a valid Bearer token on every endpoint except register, login and health; a missing, malformed, wrongly signed or expired token yields 401 UNAUTHORIZED.	Must	Implemented	auth.api.test.js
FR-AUTH-08	Rate-limit /api/auth/* to a configurable maximum per minute per client (default 20) and answer excess requests with 429 RATE_LIMITED.	Should	Implemented	config in app.js (no automated test yet)

3.2 Groups and membership (GRP)

Description. A group is the unit of sharing. Its creator becomes Admin; only Admins add people.

Endpoints: GET/POST /api/groups, GET /api/groups/:id, POST /api/groups/:id/members.

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-GRP-01	Allow an authenticated user to create a group with name (1-100 characters) and optional description (max 500); the creator shall be added as a member with role ADMIN in the same operation.	Must	Implemented	groups.test.js
FR-GRP-02	List the groups the caller belongs to, newest first, each with its members (id, name, e-mail, role).	Must	Implemented	groups.api.test.js
FR-GRP-03	Return a group's details only to its members; a non-member receives 403 FORBIDDEN; an unknown id yields 404 NOT_FOUND.	Must	Implemented	groups.api.test.js
FR-GRP-04	Allow an Admin to add an existing registered user by e-mail as a MEMBER. Non-admin: 403 FORBIDDEN; unregistered e-mail: 404 USER_NOT_FOUND; already a member: 409 DUPLICATE_MEMBER.	Must	Implemented	groups.test.js
FR-GRP-05	Enforce at most one membership per (group, user) pair at database level.	Must	Implemented	groups.test.js

3.3 Expenses (EXP)

Description. An expense records that one member paid an amount on a date for a category, shared by chosen participants under a split rule. **Endpoints:** GET/POST /api/groups/:id/expenses, PUT/DELETE /api/expenses/:id.

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-EXP-01	Allow a group member to record an expense with description (1-200 chars), amountPaise (positive integer), category (1-50 chars), date (ISO-8601 date-time), paidById , splitType (EQUAL, EXACT or PERCENT) and participants (at least one).	Must	Implemented	expenses.api.test.js
FR-EXP-02	Reject with 400 INVALID_MEMBER any expense whose payer or any participant is not a member of the group.	Must	Implemented	expenses.api.test.js
FR-EXP-03	Reject duplicate participants, an empty participant list, a zero or negative amount, a fractional paise amount, a blank description, a non-ISO date and an unknown split type, each with 400 VALIDATION_ERROR.	Must	Implemented	expenses.api.test.js
FR-EXP-04	Persist the expense, its split rows and its activity entry in one database transaction; if any part fails none is stored.	Must	Implemented	expenses.api.test.js (invalid update leaves expense unchanged)
FR-EXP-05	List a group's expenses, most recent date first, each with its splits and payer; members only (403 otherwise).	Must	Implemented	expenses.api.test.js
FR-EXP-06	Allow the expense's creator or a group Admin to edit it; recalculate and replace all its splits; others receive 403 FORBIDDEN.	Should	Implemented	expenses.test.js
FR-EXP-07	Allow the creator or an Admin to delete an expense; others receive 403; success returns 204.	Should	Implemented	expenses.test.js
FR-EXP-08	Record the creating user (createdById) separately from the payer (paidById) and keep created/updated timestamps.	Should	Implemented	expenses.api.test.js

3.4 Split rules (SPL)

Description. Turns an amount and participant input into per-person shares in paise. Implemented in the pure module services/splits.js. Worked examples are in Section 7.4.

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-SPL-01	EQUAL: with n participants, each share shall be floor(amount / n) paise; the remainder $r = \text{amount} \bmod n$ shall be distributed one paise each to the r participants with the lowest user ids . The result shall not depend on the order in which participants are supplied.	Must	Implemented	services.test.js
FR-SPL-02	EXACT: every sharePaise shall be a non-negative integer and the shares shall sum exactly to the amount; otherwise reject with 400 INVALID_SPLIT.	Must	Implemented	services.test.js
FR-SPL-03	PERCENT: every percentage shall be a non-negative whole number and the percentages shall total exactly 100; otherwise reject with 400 INVALID_SPLIT. Each share is floor(amount x percent / 100); leftover paise are given one each to the lowest user ids.	Must	Implemented	services.test.js
FR-SPL-04	For every stored expense the sum of its split shares shall equal its amount exactly (no lost or extra paise).	Must	Implemented	services.test.js, expenses.api.test.js
FR-SPL-05	Reject any split type other than EQUAL, EXACT or PERCENT.	Must	Implemented	services.test.js
FR-SPL-06	For PERCENT expenses store the entered percentage alongside the derived share.	Should	Implemented	expenses.api.test.js

3.5 Balances (BAL)

Description. Computes each member's position from the raw records. **Endpoint:** GET /api/groups/:id/balances.

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-BAL-01	Compute each member's net balance as total paid - total owed (sum of that member's shares), plus settlements the member paid, minus settlements the member received.	Must	Implemented	services.test.js
FR-BAL-02	Derive balances on demand from expense and settlement records; balances shall not be stored.	Must	Implemented	services.test.js
FR-BAL-03	The sum of all members' net balances in a group shall be zero; if it is not, the computation shall fail with an error instead of returning wrong data.	Must	Implemented	services.test.js
FR-BAL-04	Return every current member, including members with no records (balance 0).	Must	Implemented	services.test.js
FR-BAL-05	Interpret a positive balance as "the group owes this member" and a negative balance as "this member owes the group".	Must	Implemented	groupData.api.test.js

3.6 Debt simplification (DBT)

Description. Produces a short list of suggested transfers. It is returned together with the balances (simplifiedSettlements).

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-DBT-01	Return with the balances a list of suggested transfers (from, to, amountPaise) derived from the net balances.	Must	Implemented	groupData.api.test.js
FR-DBT-02	Compute the list greedily: repeatedly pair the debtor with the largest debt and the creditor with the largest credit, transfer the smaller of the two amounts, and remove any party reaching zero. Ties in amount shall be broken in favour of the lower user id.	Must	Implemented	services.test.js
FR-DBT-03	The output shall be deterministic, contain at most n-1 transfers (n = number of members with a non-zero balance), and applying all transfers shall bring every balance to zero.	Must	Implemented	services.test.js
FR-DBT-04	Suggested transfers are advisory: they shall not change any stored data until a member records them as settlements.	Must	Implemented	by design: balances endpoint is read-only

3.7 Settlements (SET)

Description. Records that a real payment took place. **Endpoint:** POST /api/groups/:id/settlements.

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-SET-01	Allow a group member to record a settlement with fromUserId , toUserId , amountPaise (positive integer) and optional note (max 300 chars); respond 201.	Should	Implemented	groupData.api.test.js
FR-SET-02	Reject with 400 a settlement whose two users are identical, or where either user is not a member of the group.	Should	Implemented	groupData.api.test.js
FR-SET-03	A settlement shall adjust both parties' balances without modifying or deleting any expense, so expense history remains intact for audit.	Should	Implemented	services.test.js
FR-SET-04	Store the settlement and its activity entry atomically.	Should	Implemented	groupData.api.test.js

3.8 Activity feed (ACT) and Dashboard (DSH)

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-ACT-01	Write an activity entry (actor, type, message, timestamp) for every expense creation, update and deletion and every settlement (types EXPENSE_CREATED, EXPENSE_UPDATED, EXPENSE_DELETED, SETTLEMENT_RECORDED).	Should	Implemented	expenses.api.test.js, groupData.test.js
FR-ACT-02	Return a group's activity newest first to its members only (403 otherwise).	Should	Implemented	groupData.api.test.js
FR-ACT-03	Activity entries shall be append-only: no endpoint edits or deletes them.	Should	Implemented	design (no route exists)
FR-DSH-01	Return the caller's cross-group totals: youOwePaise (sum of the caller's negative group balances, as a positive number) and youAreOwedPaise (sum of positive group balances).	Should	Implemented	groupData.api.test.js

3.9 Common behaviour (COM) and User interface (UI)

ID	Requirement (the system shall ...)	Pri.	Status	Verified by
FR-COM-01	Validate every input before it is persisted. Validation failures shall return 400 with the body { error: { code: "VALIDATION_ERROR", message } } naming the first failing field. Path ids shall be positive integers.	Must	Implemented	errors.test.js
FR-COM-02	All failures shall use the uniform error body { error: { code, message } }. Unexpected server faults shall return 500 INTERNAL_ERROR with a generic message that leaks no internals.	Must	Implemented	errors.test.js
FR-COM-03	Expose GET /api/health returning { ok: true } for deployment health checks.	Should	Implemented	auth.test.js
FR-UI-01	Provide SPA screens for: login and registration; dashboard (totals, group list, create group); group detail with tabs Expenses , Balances and Activity ; add-member, add-expense, edit-expense and record-settlement forms.	Must	Implemented	App.test.jsx
FR-UI-02	Display all amounts as Indian-rupee currency (en-IN formatting) converted from paise; accept rupee input and convert to integer paise before sending.	Must	Implemented	money.test.js
FR-UI-03	Offer Edit/Delete on an expense only to its creator or a group Admin, ask for confirmation before deleting, and show server error messages to the user.	Should	Implemented	App.test.jsx

4. External Interface Requirements

4.1 User interfaces

The SPA has been prototyped in HTML/PNG form (repository folder designs/): Authentication, Dashboard, Group detail (states), Expenses tab, Balances and settlement (two variants), and Activity history feed. General UI requirements:

- **NFR-linked:** forms show inline error text from the API's message field; loading states are shown on first load and not re-flashed on refresh.
- Navigation: Dashboard -> Group detail -> tabs (Expenses / Balances / Activity); a back control returns to the group list.
- Split entry: a split-type selector switches the participant fields between equal (checkbox list), exact (rupee amount per person) and percentage (whole-number percent per person).
- Layout is responsive (Tailwind utility classes); minimum supported viewport is a typical phone width.

4.2 Software interfaces

Interface	Description
Browser -> API	HTTP/1.1 with JSON bodies; Authorization: Bearer <JWT> header; CORS restricted to CORS_ORIGIN (default http://localhost:5173).
API -> Database	Prisma Client over TCP to PostgreSQL 16 using DATABASE_URL; schema managed by Prisma migrations (init migration 20260929175142).
Fonts / CDN	The UI may load Google Fonts and the Tailwind CDN script; the Content-Security-Policy allows only these hosts plus self.
Docker Compose	Services: db (postgres:16-alpine with health check), migrate (prisma migrate deploy, run once), app (Express + built SPA on port 3001).

4.3 Hardware and communications interfaces

No special hardware. The system requires TCP/IP connectivity between browser and server (port 3001 in the container deployment; 5173 for the Vite dev server) and between server and database (port 5432). All data exchange is JSON.

4.4 REST API summary

Method and path	Auth	Purpose	Success	Main errors
POST /api/auth/register	-	Create account (FR-AUTH-01..03)	201	400, 409, 429
POST /api/auth/login	-	Obtain token (FR-AUTH-04, 05)	200	400, 401, 429
GET /api/auth/me	Yes	Current user (FR-AUTH-06)	200	401
GET /api/groups	Yes	List my groups (FR-GRP-02)	200	401
POST /api/groups	Yes	Create group (FR-GRP-01)	201	400, 401
GET /api/groups/:id	Yes	Group detail (FR-GRP-03)	200	400, 403, 404
POST /api/groups/:id/members	Admin	Add member by e-mail (FR-GRP-04)	201	400, 403, 404, 409
GET /api/groups/:id/expenses	Member	List expenses (FR-EXP-05)	200	403
POST /api/groups/:id/expenses	Member	Add expense (FR-EXP-01..04)	201	400, 403
PUT /api/expenses/:id	Creator/Admin	Edit expense (FR-EXP-06)	200	400, 403, 404
DELETE /api/expenses/:id	Creator/Admin	Delete expense (FR-EXP-07)	204	403, 404
GET /api/groups/:id/balances	Member	Balances + suggested transfers (FR-BAL, FR-DBT)	200	403
POST /api/groups/:id/settlements	Member	Record settlement (FR-SET-01..04)	201	400, 403
GET /api/groups/:id/activity	Member	Activity feed (FR-ACT-02)	200	403
GET /api/dashboard/summary	Yes	Cross-group totals (FR-DSH-01)	200	401
GET /api/health	-	Liveness (FR-COM-03)	200	-

4.5 Error contract

Every error uses the body { "error": { "code": "...", "message": "..." } }.

HTTP	Code	When
400	VALIDATION_ERROR	Schema/format failure or bad id (message names the first failing field).
400	INVALID_SPLIT	Exact shares do not sum to amount; percentages do not total 100; unsupported split.
400	INVALID_MEMBER	Payer, participant or settlement party is not a group member.
401	UNAUTHORIZED / INVALID_CREDENTIALS	Missing, invalid or expired token; wrong e-mail or password.
403	FORBIDDEN	Caller is not a member, not an Admin, or not the creator/Admin of the expense.
404	NOT_FOUND / USER_NOT_FOUND	Group or expense does not exist; e-mail to add is not registered.
409	DUPLICATE_EMAIL / DUPLICATE_MEMBER	E-mail already registered; user already in group.
429	RATE_LIMITED	Too many auth attempts within one minute.
500	INTERNAL_ERROR	Unexpected fault; generic message only.

5. Non-Functional Requirements

Each non-functional requirement is stated with a measurable criterion and a verification method so that it can be checked, not merely admired.

5.1 Performance

ID	Requirement	Measure / verification
NFR-PERF-01	The balances view shall load within 2 seconds for a group of up to 50 members and 1,000 expenses.	Timed API call on seeded data of that size (target; measured during final phase).
NFR-PERF-02	Ordinary read/write API calls (login, add expense, list groups) shall respond within 1 second under single-user load on a typical laptop.	Manual timing; automated suite completes 157 server tests quickly against Docker PostgreSQL.

5.2 Security

ID	Requirement	Measure / verification
NFR-SEC-01	Passwords shall be stored only as salted bcrypt hashes with cost factor ≥ 10 and never returned in any response.	auth.api.test.js: hash stored, never returned.
NFR-SEC-02	All group data shall be accessible only to members of that group; every group-scoped endpoint shall check membership on the server.	Authorization tests: non-member gets 403 on list/add/balances/activity.
NFR-SEC-03	Authentication shall use signed JWTs (7-day expiry); tokens signed with another secret or expired shall be rejected.	auth.api.test.js.
NFR-SEC-04	The server shall set security headers (Helmet) with a restrictive Content-Security-Policy and restrict CORS to the configured origin.	Code review of app.js; header inspection.
NFR-SEC-05	Login failures shall not reveal whether an e-mail exists; auth endpoints shall be rate-limited.	auth.api.test.js; rate-limiter configuration.
NFR-SEC-06	Inputs shall be validated server-side (Zod) and database access shall go through Prisma's parameterised queries.	Validation tests; code review.
NFR-SEC-07	No credentials or secrets shall be committed; production deployments shall set a strong JWT_SECRET (see Section 11.3, item 6).	Secrets scan before submission; .env.example only.

5.3 Reliability and data integrity

ID	Requirement	Measure / verification
NFR-REL-01	Balance computation shall be deterministic: identical inputs always yield identical outputs (including rounding and tie-breaking).	services.test.js determinism tests.
NFR-REL-02	Money shall be integer paise end to end; the shares of an expense shall always sum to its amount; balances in a group shall always sum to zero.	Property-style tests and the run-time invariant check (FR-BAL-03).
NFR-REL-03	Multi-row writes (expense + splits + activity; settlement + activity) shall be atomic.	Transactional implementation; expenses.api.test.js (invalid update leaves expense unchanged).
NFR-REL-04	Deleting a group shall cascade to its memberships, expenses, splits, settlements and activity (no orphans).	Schema constraints (onDelete: Cascade).

5.4 Usability

ID	Requirement	Measure / verification
NFR-USA-01	A first-time user shall be able to register, create a group, add an expense and see balances without instructions.	Demo walkthrough with a new user (demo script).
NFR-USA-02	Validation and server errors shall be shown next to the form in plain language.	UI tests; manual inspection.
NFR-USA-03	Currency shall be displayed in Indian format (INR, two decimals) everywhere.	money.test.js.

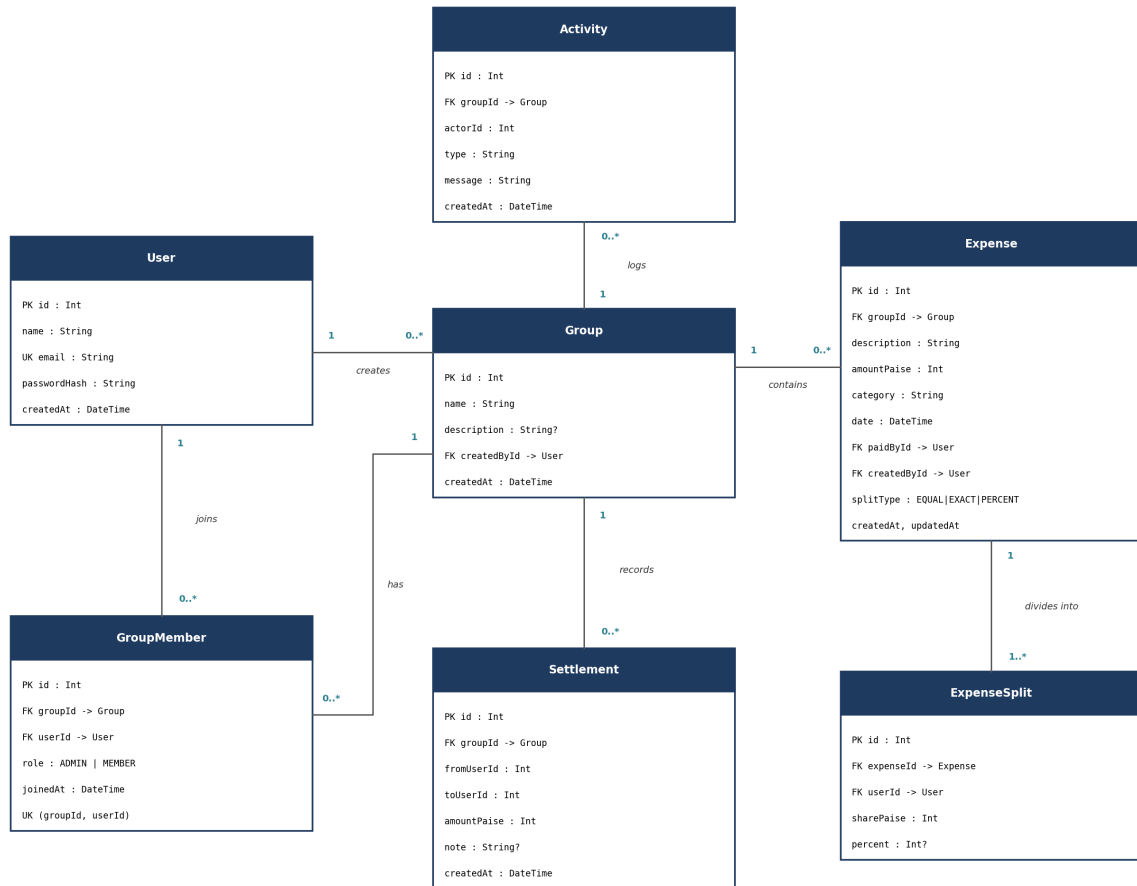
5.5 Maintainability, testability and portability

ID	Requirement	Measure / verification
NFR-MNT-01	Business logic shall be separated from UI and data access (layers: routes -> controllers -> services -> Prisma) to give high cohesion and low coupling.	Architecture review (Figure 2).
NFR-TST-01	Split, balance and simplification logic shall be pure functions covered by unit tests; API behaviour shall be covered by integration tests against an isolated test database.	Latest verified run (30 Sep 2026): 179 tests pass (157 server + 22 client); lint and production build also pass.
NFR-POR-01	The system shall run identically on any host with Docker; PostgreSQL 16 in a container gives a reproducible environment.	docker compose up; README instructions.
NFR-MNT-02	Requirement changes shall be recorded with reason, impact and approval, and this SRS and its traceability links updated.	Revision history; change log (Section 11.3).

6. Data Requirements

6.1 Entity-relationship model

The logical data model has seven entities. *ExpenseSplit* normalises per-person liability; balances are computed, never stored. Deleting a Group cascades to GroupMember, Expense (and its ExpenseSplit rows), Settlement and Activity.



User is also referenced by id from Expense.paidById / createdById, ExpenseSplit.userId, Settlement.fromUserId / toUserId and Activity.actorId.

Figure 3 - Entity-relationship diagram (matches server/prisma/schema.prisma).

6.2 Data dictionary

Types are Prisma types; **Int** money fields are paise. PK = primary key, FK = foreign key, UK = unique.

User

Field	Type	Constraint / meaning
id	Int	PK, auto-increment
name	String	Display name, 1-100 chars (validated at API)
email	String	UK; trimmed; used as login identifier
passwordHash	String	bcrypt hash; never returned
createdAt	DateTime	Default now()

Group

Field	Type	Constraint / meaning
id	Int	PK
name	String	1-100 chars
description	String?	Optional, max 500 chars
createdById	Int	FK -> User (creator)
createdAt	DateTime	Default now()

GroupMember

Field	Type	Constraint / meaning
id	Int	PK
groupId, userId	Int	FKs; UK (groupId, userId); cascade on delete
role	enum Role	ADMIN or MEMBER (default MEMBER)
joinedAt	DateTime	Default now()

Expense

Field	Type	Constraint / meaning
id	Int	PK
groupId	Int	FK -> Group, cascade
description	String	1-200 chars
amountPaise	Int	Positive integer paise
category	String	1-50 chars
date	DateTime	When the expense occurred (ISO-8601)
paidById	Int	FK -> User: who paid
createdById	Int	FK -> User: who entered it
splitType	enum SplitType	EQUAL, EXACT or PERCENT
createdAt / updatedAt	DateTime	Maintained automatically

ExpenseSplit

Field	Type	Constraint / meaning
id	Int	PK
expenseId	Int	FK -> Expense, cascade
userId	Int	FK -> User: the participant
sharePaise	Int	This participant's share; shares of one expense sum to amountPaise
percent	Int?	Whole percent, only for PERCENT splits

Settlement

Field	Type	Constraint / meaning
id	Int	PK
groupId	Int	FK -> Group, cascade
fromUserId	Int	Member who paid
toUserId	Int	Member who received; must differ from fromUserId
amountPaise	Int	Positive integer paise
note	String?	Optional, max 300 chars
createdAt	DateTime	Default now()

Activity

Field	Type	Constraint / meaning
id	Int	PK
groupId	Int	FK -> Group, cascade
actorId	Int	User who performed the action
type	String	EXPENSE_CREATED / EXPENSE_UPDATED / EXPENSE_DELETED / SETTLEMENT_RECORDED
message	String	Human-readable text, e.g. "Added expense: Dinner"
createdAt	DateTime	Default now()

6.3 Data flow dictionary (derived data)

Data flow	Composition
Net balance (per member)	paidTotal - shareTotal + settlementsPaid - settlementsReceived, in paise.
Suggested transfer	{ from: userId, to: userId, amount: paise } - one element of simplifiedSettlements.
Dashboard summary	{ youOwePaise, youAreOwedPaise } aggregated over all groups of the caller.
Public user	{ id, name, email } - the only user shape ever returned to clients.
Token	JWT payload { id, email } signed with JWT_SECRET, expiry 7 days.

6.4 Data rules and retention

- Amounts are integers; no decimal or floating type is used for money.
- Expenses and settlements are stored in full; balances are recomputed from them. Deleting an expense removes its rows, but the activity feed keeps a permanent entry.
- Personal data held: name, e-mail, password hash. No payment card or bank details are ever collected.

7. Analysis Models

7.1 Use-case diagram

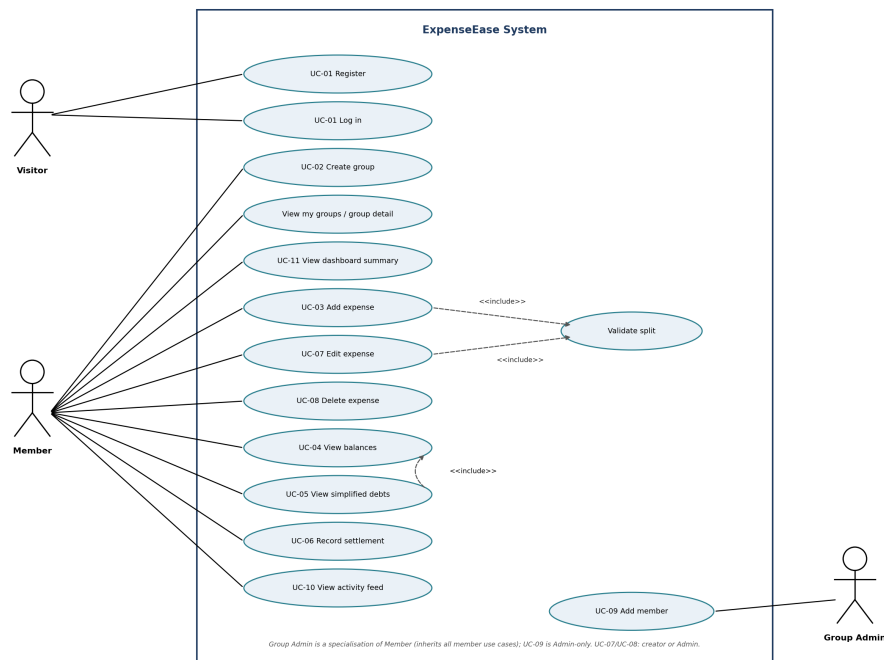


Figure 4 - Use-case diagram. Actors: Visitor, Member, Group Admin (a specialisation of Member).

7.2 Use-case descriptions

Six core use cases (UC-01 to UC-06) were defined in the mid-semester use-case document; UC-07 to UC-11 cover expense correction, membership and the read-only views that the implementation also provides.

Use case	UC-01 Register / Log in
Primary actor	Visitor
Preconditions	Visitor has valid details / an existing account.
Trigger	Visitor submits the registration or login form.
Main flow	1. Visitor enters name, e-mail, password (register) or e-mail, password (login). 2. System validates input. 3. System creates the account (register) or verifies the password hash (login). 4. System returns the public user and a 7-day token. 5. SPA stores the token and opens the dashboard.
Alternative / exception flows	A1: invalid format -> 400 with field message. A2: e-mail exists -> 409 DUPLICATE_EMAIL. A3: wrong credentials -> 401 INVALID_CREDENTIALS. A4: too many attempts -> 429.
Postconditions	Authenticated session exists.
Requirements	FR-AUTH-01..08

Use case	UC-02 Create group
Primary actor	Member (authenticated user)
Preconditions	User is authenticated.
Trigger	User submits the create-group form.
Main flow	1. User enters group name and optional description. 2. System validates. 3. System creates the group and adds the user as ADMIN in one operation. 4. Group appears in the user's list.
Alternative / exception flows	A1: empty/too-long name -> 400.
Postconditions	New group exists; creator is Admin.
Requirements	FR-GRP-01, 02

Use case	UC-03 Add expense
Primary actor	Member
Preconditions	User is a member of the group; payer and participants are members.
Trigger	User submits the add-expense form.
Main flow	1. User enters description, amount, category, date, payer, split type and participants (and shares/percentages where needed). 2. System validates input and membership. 3. System calculates shares by the chosen rule (Section 7.4). 4. System stores expense, split rows and activity entry in one transaction. 5. Expense list and balances refresh.
Alternative / exception flows	A1: exact shares do not sum to amount, or percentages do not total 100 -> 400 INVALID_SPLIT. A2: payer/participant not a member -> 400 INVALID_MEMBER. A3: caller not a member -> 403. A4: unauthenticated -> 401.
Postconditions	Shares sum exactly to the stored amount; activity recorded.
Requirements	FR-EXP-01..05, FR-SPL-01..06

Use case	UC-04 View balances
Primary actor	Member
Preconditions	User is a member.
Trigger	User opens the Balances tab.
Main flow	1. System loads members, expenses and settlements. 2. System computes net balance per member and checks the sum is zero. 3. System displays each member's balance (positive = is owed, negative = owes).
Alternative / exception flows	A1: invariant fails -> 500 with generic message (should never occur with valid data).
Postconditions	Computed balances shown; nothing stored.
Requirements	FR-BAL-01..05

Use case	UC-05 View simplified debts
Primary actor	Member
Preconditions	Balances sum to zero (UC-04 succeeded).
Trigger	Balances tab is displayed.
Main flow	1. System runs the greedy simplifier on the net balances. 2. System lists the suggested transfers, at most n-1.
Alternative / exception flows	A1: everyone settled -> empty list.
Postconditions	Deterministic suggested transfers shown; advisory only.
Requirements	FR-DBT-01..04

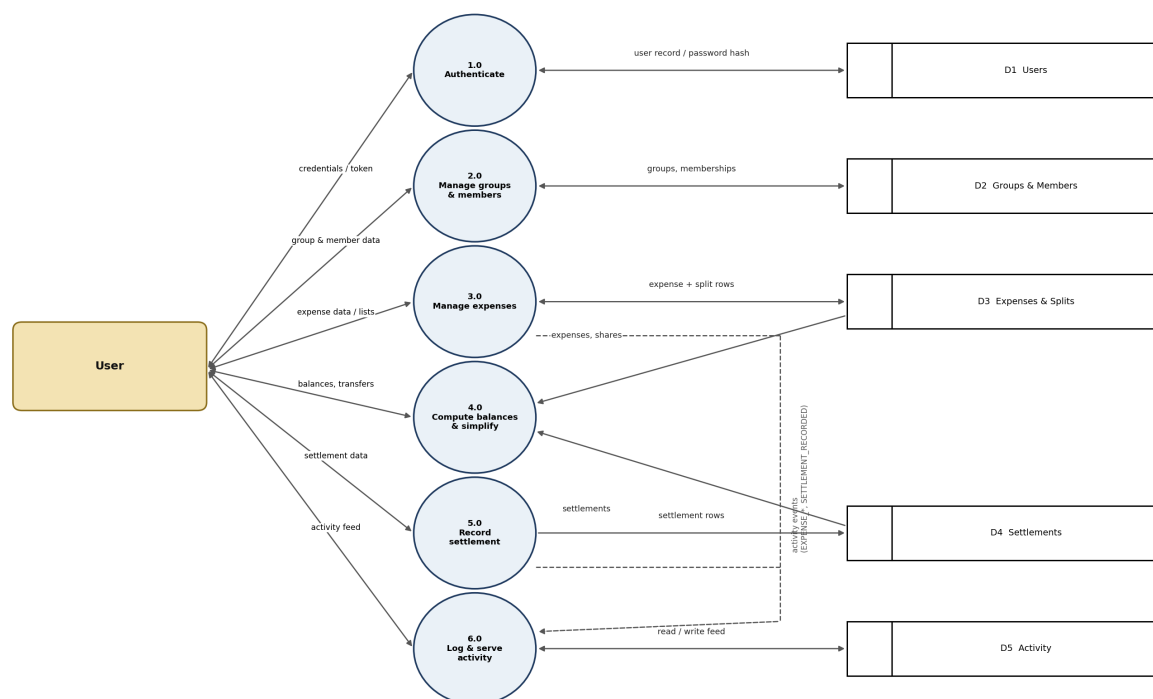
Use case	UC-06 Record settlement
Primary actor	Member
Preconditions	Both parties are members.
Trigger	User submits the record-settlement form.
Main flow	1. User selects payer, receiver, amount and optional note (or picks a suggested transfer). 2. System validates parties differ and are members. 3. System stores the settlement and an activity entry atomically. 4. Balances update.
Alternative / exception flows	A1: same user for both parties or non-member -> 400. A2: non-positive amount -> 400.
Postconditions	Settlement affects computed balances; expenses untouched.
Requirements	FR-SET-01..04
Use case	UC-07 Edit expense
Primary actor	Member (creator) or Group Admin
Preconditions	Expense exists; caller is its creator or an Admin.
Trigger	User submits the edit form.
Main flow	1. System checks role/creator. 2. System validates new data and recalculates splits. 3. System replaces the splits, updates the expense and writes an EXPENSE_UPDATED activity in one transaction.
Alternative / exception flows	A1: other member -> 403 FORBIDDEN. A2: unknown id -> 404.
Postconditions	Expense reflects the correction; feed shows the change.
Requirements	FR-EXP-06
Use case	UC-08 Delete expense
Primary actor	Member (creator) or Group Admin
Preconditions	Expense exists; caller is its creator or an Admin.
Trigger	User confirms deletion.
Main flow	1. System checks role/creator. 2. System deletes the expense (splits cascade) and writes EXPENSE_DELETED activity atomically. 3. Balances no longer include it.
Alternative / exception flows	A1: other member -> 403. A2: unknown id -> 404.
Postconditions	Expense removed; audit entry remains.
Requirements	FR-EXP-07
Use case	UC-09 Add member
Primary actor	Group Admin
Preconditions	User is an Admin of the group.
Trigger	Admin submits a member's e-mail.
Main flow	1. System verifies Admin role. 2. System finds the registered user by e-mail. 3. System creates a MEMBER membership.
Alternative / exception flows	A1: caller not Admin -> 403. A2: e-mail not registered -> 404 USER_NOT_FOUND. A3: already a member -> 409 DUPLICATE_MEMBER.
Postconditions	New member can access the group.
Requirements	FR-GRP-04, 05

Use case	UC-10 View activity feed
Primary actor	Member
Preconditions	User is a member.
Trigger	User opens the Activity tab.
Main flow	1. System returns the group's activity newest first. 2. SPA renders the feed.
Alternative / exception flows	A1: non-member -> 403.
Postconditions	Read-only.
Requirements	FR-ACT-01..03

Use case	UC-11 View dashboard summary
Primary actor	Member
Preconditions	User is authenticated.
Trigger	User opens the dashboard.
Main flow	1. System computes the caller's balance in every group they belong to. 2. System sums negative balances (you owe) and positive balances (you are owed). 3. SPA shows the two totals.
Alternative / exception flows	A1: user has no groups -> both totals are 0.
Postconditions	Read-only.
Requirements	FR-DSH-01

7.3 Data-flow diagram (Level 1)

Figure 5 decomposes the system of Figure 1 into six processes and five data stores. Every process except authentication verifies group membership against store D2 before touching group data.



Processes 3.0-6.0 also read D2 to verify that the caller is a member of the group (403 FORBIDDEN otherwise).

Figure 5 - DFD Level 1.

7.4 Key algorithms and worked examples

Equal split (FR-SPL-01)

$\text{share}(i) = \text{floor}(A / n) + (1 \text{ if } i \text{ is among the first } (A \bmod n) \text{ participants by ascending user id, else } 0).$

Example: $A = 10,000$ paise among users 1, 2, 3 $\rightarrow$ base 3,333, remainder 1 $\rightarrow$ **3,334 / 3,333 / 3,333** (sum 10,000).

Percentage split (FR-SPL-03)

Example: $A = 10,001$ paise, users 1, 2, 3 with 33 %, 33 %, 34 %. Floors: $3,300 / 3,300 / 3,400 = 10,000$; one paise left over $\rightarrow$ user 1 gets it: **3,301 / 3,300 / 3,400** (sum 10,001). Entering 50 % and 40 % is rejected because the total is not 100.

Net balance (FR-BAL-01)

Ana pays 3,000 for Ana, Ben and Chi (equal, 1,000 each) and Ben pays 600 for Ana and Ben (300 each). Net: Ana = $3,000 - 1,300 = +1,700$; Ben = $600 - 1,300 = -700$; Chi = $0 - 1,000 = -1,000$. Sum = 0. If Chi settles 1,000 to Ana: Chi = 0, Ana = +700.

Debt simplification (FR-DBT-02, 03)

```
repeat while a debtor and a creditor remain:
    d = debtor with largest debt      (tie  $\rightarrow$  lower user id)
    c = creditor with largest credit (tie  $\rightarrow$  lower user id)
    t = min(d.amount, c.amount)
    emit transfer d  $\rightarrow$  c of t
    subtract t from both; drop whichever reached zero
```

Example: balances A +6,000; B +2,000; C -4,000; D -4,000. C (id 3) and D (id 4) tie; C pays A 4,000 (A has 2,000 left); D (4,000) pays A 2,000 (A ties with B, lower id first) then B 2,000. Result: **C $\rightarrow$ A 4,000; D $\rightarrow$ A 2,000; D $\rightarrow$ B 2,000** - three transfers, $n-1$ for four people, instead of up to four pairwise payments. Finding the true minimum is NP-hard; the greedy method is a deliberate, deterministic approximation that never exceeds $n-1$ transfers.

8. Prioritisation of Requirements

Requirements were ranked with MoSCoW. **Must** items form the minimum end-to-end product (register -> group -> expense -> exact balances); **Should** items improve correctness handling and completeness of the workflow; **Could** items are candidates for the remaining semester.

Priority	Requirements	Rationale
Must	FR-AUTH-01..05, 07; FR-GRP-01..05; FR-EXP-01..05; FR-SPL-01..05; FR-BAL-01..05; FR-DBT-01..04; FR-COM-01, 02; FR-UI-01, 02; NFR-SEC-01..03; NFR-REL-01, 02	Without them the product cannot record a shared expense and report correct balances securely.
Should	FR-AUTH-06, 08; FR-EXP-06..08; FR-SPL-06; FR-SET-01..04; FR-ACT-01..03; FR-DSH-01; FR-COM-03; FR-UI-03; NFR-PERF-01; NFR-USA	Real groups need corrections, settlements and history, but a manual workaround exists.
Could (future)	Group edit/delete; remove member; password change/reset; token refresh and logout revocation; settlement suggestion buttons pre-filling the form; export to CSV; category summaries and charts; search/filter of expenses	Valuable but not needed for the mid-semester end-to-end demonstration.

Requirement change from the earlier draft

Draft 0.1 mapped requirements as FR-01 to FR-06. They are retained as groups: FR-01 -> FR-AUTH; FR-02 -> FR-GRP; FR-03 -> FR-EXP + FR-SPL; FR-04 -> FR-BAL; FR-05 -> FR-DBT; FR-06 -> FR-SET.

9. Features Excluded and Deferred

Excluded	Reason
Payment processing or bank/UPI integration	Settlements are recorded manually; regulatory and security scope is out of range for the course project.
Receipt OCR, attachments	Extra complexity, not needed to demonstrate the core value.
Multi-currency and exchange rates	The product is INR-only; integer-paise arithmetic is simplest and exact.
Recurring expenses, reminders, e-mail/push notifications	Require scheduling and external services.
Social login, e-mail verification, password reset	Authentication is deliberately limited to e-mail and password with JWT.
Native mobile apps	The responsive web SPA covers mobile browsers.
Editing or deleting groups; removing members; changing roles	Not implemented in this increment (see Section 11.3).

10. Requirements Traceability Matrix

Traceability lets us see that every requirement is addressed in every phase, and to assess the impact of a change. Files are relative to server/ unless prefixed by client/.

Requirement(s)	User story (US)	Use case	Design / code element	Tests
FR-AUTH-01..08	US-1, US-2	UC-01	routes/auth.js, controllers/auth.js, middleware/auth.js, User model	auth.api.test.js, errors.test.js
FR-GRP-01..05	US-3, US-4, US-5	UC-02, UC-09	routes/groups.js, services/groups.js, Group + GroupMember models	groups.test.js, groups.api.test.js
FR-EXP-01..05, 08	US-10	UC-03	controllers/expenses.js (Zod), services/expenses.js, Expense model	expenses.api.test.js, expenses.test.js
FR-EXP-06, 07	US-11, US-12, US-13	UC-07, UC-08	services/expenses.js (creator/Admin check)	expenses.test.js
FR-SPL-01	US-6, US-9	UC-03	services/splits.js equalSplits()	services.test.js
FR-SPL-02	US-7	UC-03	services/splits.js exactSplits()	services.test.js
FR-SPL-03, 06	US-8, US-9	UC-03	services/splits.js percentSplits(), ExpenseSplit.percent	services.test.js, expenses.api.test.js
FR-BAL-01..05	US-14	UC-04	services/balances.js computeBalances()	services.test.js, groupData.api.test.js
FR-DBT-01..04	US-15	UC-05	services/debtSimplifier.js simplifyDebts()	services.test.js
FR-SET-01..04	US-16	UC-06	controllers/groupData.js, services/groupData.js, Settlement model	groupData.api.test.js, groupData.test.js
FR-ACT-01..03	US-17	UC-10	services/groupData.js activity(), Activity model	groupData.api.test.js
FR-DSH-01	US-18	UC-11	services/groupData.js dashboard()	groupData.api.test.js
FR-COM-01..03	(all)	(all)	middleware/errors.js, Zod schemas, app.js	errors.test.js, auth.test.js
FR-UI-01..03	US-5, US-10, US-14, US-16, US-17	UC-01..11	client/src/App.jsx, utils/money.js, designs/*	client/src/App.test.jsx, money.test.js
NFR-SEC-01..07	US-2	UC-01	bcrypt, jsonwebtoken, helmet, cors, express-rate-limit	auth.api.test.js; review
NFR-REL-01..04	US-9, US-14	UC-03, UC-04	integer paise, \$transaction, onDelete: Cascade	services.test.js

Legacy mapping: the three rows of the draft-0.1 traceability table (FR-03, FR-04, FR-05 -> services.test.js) are subsumed by the rows FR-SPL, FR-BAL and FR-DBT above.

11. Requirements Validation Record

11.1 Quality checklist

The SRS was reviewed against the attributes of a good specification. Verification asks whether we built it right; validation asks whether we built the right thing (through the demo walkthrough).

Attribute	Check performed	Result
Complete	Every stakeholder function in Section 2.3 has requirements; interfaces, data, errors and NFRs are covered.	Met; gaps listed in 11.3
Consistent	No two requirements contradict; error codes and limits are the same in Sections 3, 4 and 6 and match the Zod schemas.	Met
Unambiguous	Rounding, tie-breaking, roles and sign conventions are stated explicitly (FR-SPL-01, FR-DBT-02, FR-BAL-05).	Met
Verifiable	Each requirement names a test or review; NFRs carry measures.	Met

Attribute	Check performed	Result
Modifiable / traceable	Unique IDs, module grouping, RTM in Section 10.	Met
Black-box	Requirements describe externally visible behaviour; code names appear only in Sections 4, 6, 10.	Met

11.2 Verification status

Latest verified run (30 September 2026, repository root): **179 automated tests passed (157 server, 22 client)**, plus lint and production build. Server tests execute against an isolated `expenseease_test` PostgreSQL database. Remaining evidence to collect for the submission: screenshots of the command output and of the working browser flow, and the NFR-PERF-01 timing measurement.

11.3 Known limitations, open issues and change log

#	Item	Impact / plan
1	Expenses are editable and deletable. The draft SRS wording "balances computed from immutable records" was incorrect for the built system; this version specifies balances as derived from current records, with an append-only activity feed for audit.	Corrected here (FR-BAL-02, FR-ACT-03).
2	README's early sections mention SQLite; the implemented and specified database is PostgreSQL 16 (Prisma schema, Docker Compose).	README to be aligned before submission.
3	A settlement is not checked against the actual debt between the two parties; a member can record any positive amount between any two members.	Accepted for mid-sem (FR-SET-02 defines what is checked). Candidate Could-requirement.
4	The settlement activity message is generic ("Recorded settlement") and does not name the parties or amount.	Improvement candidate.
5	No group edit/delete, member removal, role change, password change/reset, refresh tokens or server-side logout.	Listed in Sections 8-9 as excluded/deferred.
6	If <code>JWT_SECRET</code> is not set the server falls back to a built-in placeholder secret. Every real deployment must set a strong secret (NFR-SEC-07); consider failing at start-up instead.	Security hardening item for the remaining semester.
7	Rate limiting on <code>/api/auth/*</code> (FR-AUTH-08) is implemented but has no automated test.	Add a test that exceeds <code>AUTH_RATE_LIMIT_MAX</code> and expects 429.
8	Balances are recomputed from all records on every request; performance beyond the NFR-PERF-01 size has not been measured.	Measure; cache only if needed.

Appendix A. Sample API exchanges

Amounts are paise (INR 1,200.00 = 120000). Ids are illustrative. The runnable demo dataset and further request bodies are in sample_data/; run `npm run seed` for local demo accounts.

A.1 Add an equal-split expense - POST /api/groups/1/expenses

```
{ "description": "Dinner", "amountPaise": 10000, "category": "Food",
  "date": "2026-09-20T19:30:00.000Z", "paidById": 1, "splitType": "EQUAL",
  "participants": [ {"userId":1}, {"userId":2}, {"userId":3} ] }
```

201 -> splits: user1 3334, user2 3333, user3 3333 (sum 10000)

A.2 Percentage split with rounding

```
{ "amountPaise": 10001, "splitType": "PERCENT",
  "participants": [ {"userId":1,"percent":33}, {"userId":2,"percent":33}, {"userId":3,"percent":34} ] }
```

201 -> 3301 / 3300 / 3400 paise percent 50 + 40 -> 400 INVALID_SPLIT

A.3 Balances and suggested transfers - GET /api/groups/1/balances

```
200 -> { "members": [ {"user":{"id":1,...},"role":"ADMIN","netPaise": 6000},
                      {"user":{"id":2,...},"role":"MEMBER","netPaise": 2000},
                      {"user":{"id":3,...},"role":"MEMBER","netPaise":-4000},
                      {"user":{"id":4,...},"role":"MEMBER","netPaise":-4000} ],
  "simplifiedSettlements": [ {"from":3,"to":1,"amount":4000},
                             {"from":4,"to":1,"amount":2000},
                             {"from":4,"to":2,"amount":2000} ] }
```

A.4 Record a settlement - POST /api/groups/1/settlements

```
{ "fromUserId": 3, "toUserId": 1, "amountPaise": 4000, "note": "UPI" }      201 -> { "settlement": {...} }
```

A.5 Error example

```
400 -> { "error": { "code": "INVALID_SPLIT", "message": "Exact shares must sum to total" } }
403 -> { "error": { "code": "FORBIDDEN", "message": "You are not a member of this group" } }
```

Appendix B. Requirement index

Module	IDs	Count
Authentication	FR-AUTH-01 to 08	8
Groups	FR-GRP-01 to 05	5
Expenses	FR-EXP-01 to 08	8
Split rules	FR-SPL-01 to 06	6
Balances	FR-BAL-01 to 05	5
Debt simplification	FR-DBT-01 to 04	4
Settlements	FR-SET-01 to 04	4
Activity / Dashboard	FR-ACT-01 to 03; FR-DSH-01	4
Common / UI	FR-COM-01 to 03; FR-UI-01 to 03	6
Functional total		50
Non-functional	NFR-PERF-01, 02; NFR-SEC-01 to 07; NFR-REL-01 to 04; NFR-USA-01 to 03; NFR-MNT-01, 02; NFR-TST-01; NFR-POR-01	20